

iTeam University *Tunis - Tunisia*



Projet Semestriel (S4)

Pour l'obtention de : **Ingénieur en Cybersécurité**

Thème :

Conception, déploiement et analyse de sécurité d'une application web hébergée sur AWS via la conteneurisation Docker

Élaboré par : Mohamed Salem KHYARHOUM

Encadré par : M. Raihane Modhaffer

Année Universitaire : 2025-2026

Dédicaces

À mes parents,

qui m'ont soutenu sans relâche tout au long de mes études et qui n'ont jamais cessé de croire en moi.

À mes frères et sœurs,

pour leurs encouragements constants et leur présence chaleureuse.

À mes enseignants,

qui m'ont transmis le goût du savoir et la rigueur du travail bien fait.

À mes camarades de promotion,

avec qui j'ai partagé tant d'heures de travail, de doutes et de réussites.

À tous ceux qui, de près ou de loin,

*ont contribué à la concrétisation de ce projet,
je dédie ce modeste travail.*

Mohamed Salem

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma profonde reconnaissance à toutes les personnes qui, par leur soutien, leurs conseils ou leur présence, ont contribué à l'aboutissement de ce travail.

Je remercie tout d'abord **Allah le Tout-Puissant** de m'avoir donné la santé, la patience et la volonté nécessaires pour mener à bien ce projet.

Mes plus sincères remerciements s'adressent à mon encadrant, **M. Raihane Modhaffer**, pour sa disponibilité, ses conseils éclairés et son suivi rigoureux tout au long de la réalisation de ce projet. Ses remarques pertinentes m'ont permis d'orienter mon travail dans la bonne direction et d'en améliorer la qualité.

Je tiens également à remercier l'ensemble du corps enseignant de **l'iTeam University** pour la qualité de la formation reçue durant mes années d'études, ainsi que pour l'environnement académique stimulant qu'ils ont su créer.

Mes remerciements vont aussi aux **membres du jury** qui ont accepté d'évaluer ce travail. Leurs critiques constructives constitueront une précieuse contribution à mon développement professionnel futur.

Enfin, j'adresse une pensée toute particulière à ma **famille** et à mes **amis** pour leur soutien indéfectible, leur patience et leurs encouragements, qui m'ont permis de surmonter les moments difficiles et d'aller jusqu'au bout de cette aventure.

À toutes et à tous, merci.

Mohamed Salem KHYARHOUM

Résumé

Ce projet de fin d'études porte sur la conception, le développement, le déploiement et l'analyse de sécurité d'une application web de prise de notes personnelle, baptisée **WebCyber**. L'application repose sur le *framework* Python **Flask** pour la couche métier, sur **PostgreSQL** pour la persistance des données, et sur **Nginx** comme proxy inverse assurant la terminaison TLS. L'ensemble est **conteneurisé** avec Docker et orchestré via Docker Compose, puis **déployé sur AWS** (instance EC2 t2.micro sous Ubuntu Server 22.04 LTS), avec une résolution de nom de domaine gérée par **Route 53** et un certificat HTTPS automatique fourni par **Let's Encrypt**.

Une attention particulière a été portée à la sécurité, organisée en **six couches** : pare-feu réseau (*Security Group* AWS), durcissement du système, isolation Docker, configuration TLS et en-têtes de sécurité Nginx, contrôles applicatifs (authentification, hachage PBKDF2-SHA256, protection CSRF, validation des entrées, isolation des données par utilisateur) et sécurité de la base de données. La solution déployée a ensuite été soumise à une campagne d'évaluation à l'aide de **Nmap**, **Nikto** et **testssl.sh**, dont les résultats sont analysés et discutés.

Abstract

This end-of-studies project covers the design, development, deployment and security assessment of a personal note-taking web application named **WebCyber**. The application is built on the Python **Flask** framework for business logic, **PostgreSQL** for data persistence, and **Nginx** as a reverse proxy handling TLS termination. The whole stack is **containerised** with Docker and orchestrated through Docker Compose, then **deployed on AWS** (EC2 t2.micro instance running Ubuntu Server 22.04 LTS), with DNS resolution provided by **Route 53** and HTTPS certificates automatically issued by **Let's Encrypt**.

Security has been treated as a first-class concern and structured into **six layers**: AWS Security Group network firewall, host hardening, Docker isolation, Nginx TLS and security headers, application-level controls (authentication, PBKDF2-SHA256 password hashing, CSRF protection, input validation, per-user data isolation) and database security. The deployed solution was then assessed using **Nmap**, **Nikto** and **testssl.sh**, and the results are analysed and discussed.

Table des matières

Introduction Générale	12
1. Contexte	12
2. Problématique	12
3. Objectifs	12
4. Plan du rapport	13
5. Approche méthodologique	13
5.1. Organisation des sprints	14
1 CONTEXTE GÉNÉRAL ET ÉTAT DE L'ART	15
1.1 Introduction	16
1.2 Cadre institutionnel	16
1.3 Cloud computing	16
1.3.1 Définition	16
1.3.2 Modèles de service	16
1.3.3 Le fournisseur AWS	17
1.4 Conteneurisation	17
1.4.1 Du <i>bare-metal</i> aux conteneurs	17
1.4.2 Docker et Docker Compose	17
1.4.3 Bénéfices pour ce projet	17
1.5 Architecture des applications web modernes	18
1.5.1 Le modèle 3-tiers	18
1.5.2 Le rôle du proxy inverse	18
1.6 Cybersécurité applicative	18
1.6.1 Le Top 10 OWASP	18
1.6.2 La défense en profondeur	19
1.6.3 TLS et HTTPS	19
1.7 Conclusion du chapitre	19
2 ANALYSE ET SPÉCIFICATION DES BESOINS	20
2.1 Introduction	21
2.2 Description générale du projet	21
2.3 Identification des acteurs	21
2.4 Besoins fonctionnels	22
2.4.1 Liste des besoins	22
2.4.2 Diagramme de cas d'utilisation	22
2.4.3 User Stories	23
2.4.4 Product Backlog	24
2.5 Besoins non fonctionnels	25
2.5.1 Sécurité	25
2.5.2 Disponibilité et performance	25
2.5.3 Déploiement et maintenabilité	26
2.5.4 Ergonomie	26
2.5.5 Périmètre exclu	26
2.6 Conclusion du chapitre	26
3 CONCEPTION	27

3. 1	Introduction	28
3. 2	Architecture globale	28
3. 2.1	Composants et responsabilités	29
3. 3	Infrastructure AWS	29
3. 3.1	Choix de la région	29
3. 3.2	Composants AWS retenus	29
3. 3.3	Règles du Security Group	30
3. 3.4	Estimation des coûts	31
3. 4	Architecture Docker	31
3. 4.1	Vue d'ensemble	31
3. 4.2	Spécifications des conteneurs	32
3. 4.3	Volumes persistants	32
3. 4.4	Estimation des ressources	32
3. 5	Modèle de données	32
3. 5.1	Description	33
3. 5.2	Considérations de sécurité dans le modèle	33
3. 6	Diagrammes de séquence	33
3. 6.1	Authentification	33
3. 6.2	Création d'une note	34
3. 7	Conception de la sécurité - défense en profondeur	35
3. 7.1	Couche 1 - Security Group	36
3. 7.2	Couche 2 - Système hôte	36
3. 7.3	Couche 3 - Isolation Docker	36
3. 7.4	Couche 4 - Nginx + TLS	36
3. 7.5	Couche 5 - Application Flask	37
3. 7.6	Couche 6 - Base de données	37
3. 8	Conclusion du chapitre	37
4	RÉALISATION ET DÉPLOIEMENT	38
4. 1	Introduction	39
4. 2	Environnement et outils	39
4. 2.1	Poste de développement	39
4. 2.2	Stack technologique retenue	39
4. 3	Implémentation de l'application Flask	40
4. 3.1	Organisation du code	40
4. 3.2	Modèles SQLAlchemy	40
4. 3.3	Authentification et isolation	40
4. 4	Conteneurisation	40
4. 4.1	Dockerfile applicatif	40
4. 4.2	Fichier docker-compose.yml	40
4. 5	Infrastructure as Code avec Terraform	40
4. 5.1	Motivation	40
4. 5.2	Ressources provisionnées	41
4. 6	Configuration DNS via Route 53	41
4. 7	Configuration Nginx et HTTPS	41
4. 7.1	Configuration du proxy	41
4. 7.2	Durcissement des cookies de session	42

4. 7.3	Certificat Let's Encrypt	42
4. 8	Pipeline CI/CD avec GitHub Actions	42
4. 8.1	Vue d'ensemble	42
4. 8.2	Secrets GitHub utilisés	42
4. 8.3	Bénéfices de l'approche	42
4. 9	Vue d'ensemble du flux HTTPS	43
4. 9.1	Captures d'écran	43
4. 10	Diagramme de déploiement	45
4. 11	Conclusion du chapitre	45
5	TESTS ET ANALYSE DE SÉCURITÉ	47
5. 1	Introduction	48
5. 2	Stratégie de test	48
5. 3	Tests fonctionnels	48
5. 3.1	Inscription et authentification	48
5. 3.2	Gestion des notes (CRUD)	49
5. 3.3	Test critique : isolation des données	49
5. 4	Analyse statique de l'IaC avec tfsec	49
5. 4.1	Principe et positionnement	49
5. 4.2	Résultats et décisions	49
5. 4.3	Résultat final	50
5. 5	Vérification de l'infrastructure	50
5. 5.1	Résolution DNS	50
5. 5.2	État des conteneurs	50
5. 5.3	Connectivité	50
5. 6	Scan de ports avec Nmap	51
5. 6.1	Objectif	51
5. 6.2	Résultats observés	51
5. 7	Scan applicatif Web avec Nikto	51
5. 7.1	Objectif	51
5. 7.2	Résultats typiques et interprétation	52
5. 8	Évaluation TLS	52
5. 8.1	Méthodologie	52
5. 8.2	Résultats	52
5. 9	Synthèse des audits	53
5. 10	Limites de la campagne	53
5. 11	Conclusion du chapitre	53
6	CONCLUSION GÉNÉRALE	54
6. 1	Bilan du projet	55
6. 2	Apports méthodologiques	55
6. 2.1	Retour sur la démarche agile	55
6. 3	Limites et perspectives d'évolution	56
6. 4	Conclusion	56
	Bibliographie	57

Liste des figures

Figure 1	Architecture globale - vue d'ensemble.	28
Figure 2	Infrastructure AWS détaillée.	30
Figure 3	Diagramme de classes (modèle de données).	33
Figure 4	Flux d'une requête HTTPS, des en-têtes au stockage.	43
Figure 5	Interface de connexion / inscription.	44
Figure 6	Interface de gestion des notes.	45
Figure 7	Diagramme de déploiement UML.	45

Liste des tableaux

Tableau 1	Organisation des sprints du projet WebCyber.	14
Tableau 2	Tableau des besoins fonctionnels.	22
Tableau 3	User Stories du projet WebCyber.	24
Tableau 4	Product Backlog du projet WebCyber.	25
Tableau 5	Tableau des composants et responsabilités	29
Tableau 6	Règles entrantes du Security Group `webcyber-sg`	30
Tableau 7	Estimation mensuelle (sous _free tier_ AWS).	31
Tableau 8	Spécifications des conteneurs	32
Tableau 9	Empreinte mémoire estimée des conteneurs.	32
Tableau 10	Synthèse de la pile technologique.	39
Tableau 11	Enregistrements DNS de la zone hébergée `webcyber.app`	41
Tableau 12	En-têtes de sécurité HTTP appliqués par Nginx.	41
Tableau 13	Étapes du pipeline GitHub Actions.	42
Tableau 14	Secrets configurés dans le dépôt GitHub.	42
Tableau 15	Phases de la campagne de validation.	48
Tableau 16	Tests d'inscription.	48
Tableau 17	Tests d'authentification.	48
Tableau 18	Tests CRUD sur les notes.	49
Tableau 19	Tests d'isolation par utilisateur.	49
Tableau 20	Constats tfsec sur le module Terraform et décisions associées.	50
Tableau 21	Résultats du scan Nmap.	51
Tableau 22	Synthèse des résultats Nikto.	52
Tableau 23	Synthèse de l'évaluation TLS.	52
Tableau 24	Tableau de bord de sécurité.	53

Liste des acronymes

Acronyme	Signification
ACME	<i>Automatic Certificate Management Environment (protocole Let's Encrypt)</i>
AMI	<i>Amazon Machine Image</i>
API	<i>Application Programming Interface</i>
AWS	<i>Amazon Web Services</i>
CI/CD	<i>Continuous Integration / Continuous Deployment</i>
CIDR	<i>Classless Inter-Domain Routing</i>
CRUD	<i>Create, Read, Update, Delete</i>
CSP	<i>Content Security Policy</i>
CSRF	<i>Cross-Site Request Forgery</i>
DNS	<i>Domain Name System</i>
EBS	<i>Elastic Block Store</i>
EC2	<i>Elastic Compute Cloud</i>
EIP	<i>Elastic IP Address</i>
GHA	<i>GitHub Actions</i>
GHCR	<i>GitHub Container Registry</i>
HSTS	<i>HTTP Strict Transport Security</i>
HTTP	<i>Hypertext Transfer Protocol</i>
HTTPS	<i>HTTP Secure</i>
IaaS	<i>Infrastructure as a Service</i>
IaC	<i>Infrastructure as Code</i>
IGW	<i>Internet Gateway</i>
IDOR	<i>Insecure Direct Object Reference</i>
IMDSv2	<i>Instance Metadata Service version 2</i>
IP	<i>Internet Protocol</i>
LTS	<i>Long-Term Support</i>
MVC	<i>Modèle-Vue-Contrôleur</i>
NIST	<i>National Institute of Standards and Technology</i>
ORM	<i>Object-Relational Mapping</i>
OWASP	<i>Open Web Application Security Project</i>
PaaS	<i>Platform as a Service</i>
PBKDF2	<i>Password-Based Key Derivation Function 2</i>
PFE	<i>Projet de Fin d'Études</i>
SaaS	<i>Software as a Service</i>
SAST	<i>Static Application Security Testing</i>

SG	<i>Security Group</i>
SQL	<i>Structured Query Language</i>
SSH	<i>Secure Shell</i>
SSL	<i>Secure Sockets Layer</i>
TCP	<i>Transmission Control Protocol</i>
TLD	<i>Top-Level Domain</i>
TLS	<i>Transport Layer Security</i>
URL	<i>Uniform Resource Locator</i>
UML	<i>Unified Modeling Language</i>
VPC	<i>Virtual Private Cloud</i>
WSGI	<i>Web Server Gateway Interface</i>
XSS	<i>Cross-Site Scripting</i>

Introduction Générale

1. Contexte

À l'ère de la transformation numérique, la majorité des services quotidiens — communication, gestion documentaire, commerce, administration — repose sur des applications web hébergées dans le *cloud*. Cette omniprésence du Web s'est accompagnée d'une explosion des incidents de sécurité : selon le rapport *Verizon Data Breach Investigations Report 2023*, plus de 80% des compromissions enregistrées exploitent des vulnérabilités applicatives ou des erreurs de configuration. Concevoir une application web ne se résume donc plus à coder un service fonctionnel : il faut également maîtriser son hébergement, son réseau, son cycle de déploiement et sa surface d'attaque.

Le mouvement *DevOps* et l'adoption généralisée des **conteneurs** ont profondément changé la manière de construire et de livrer ces applications. Docker offre une portabilité quasi parfaite entre les environnements de développement et de production, tandis que les fournisseurs de *cloud* comme **Amazon Web Services** (AWS) permettent de provisionner en quelques minutes des infrastructures auparavant réservées aux grands centres de données. La maîtrise de ces outils est désormais une compétence attendue de tout développeur ou administrateur système.

2. Problématique

Si les briques technologiques sont accessibles, leur *intégration sécurisée* reste un exercice difficile : combien de bases de données exposées sur Internet, combien de certificats expirés, combien d'applications vulnérables aux injections SQL ou au *cross-site scripting* ?

La problématique de ce projet peut être formulée ainsi :

Comment concevoir, développer et déployer une application web fonctionnelle dans le *cloud*, en appliquant de manière explicite et vérifiable les bonnes pratiques de cybersécurité, depuis la couche réseau jusqu'au code applicatif ?

3. Objectifs

Le projet **WebCyber** apporte une réponse pédagogique et pragmatique à cette problématique en poursuivant les objectifs suivants :

1. Concevoir une application web simple mais réaliste : une application personnelle de prise de notes (CRUD complet, authentification, isolation des données par utilisateur).

2. Conteneuriser l'ensemble des composants (Nginx, application Flask, base PostgreSQL) avec Docker et Docker Compose.
3. Déployer la solution sur une infrastructure AWS minimaliste mais représentative (EC2, Elastic IP, Security Group, Route 53).
4. Mettre en œuvre une chaîne de sécurité **en profondeur** couvrant le réseau, le système, les conteneurs, le proxy, le code applicatif et la base de données.
5. Activer HTTPS via un certificat Let's Encrypt, avec renouvellement automatisé.
6. Évaluer la posture de sécurité du déploiement à l'aide d'outils standard (Nmap, Nikto, testssl.sh) et discuter les résultats.
7. Rédiger une documentation technique exploitable permettant à un autre étudiant ou ingénieur de reproduire la solution.

4. Plan du rapport

Ce mémoire s'articule autour de cinq chapitres :

- Le **chapitre 1** pose le *contexte général* et l'*état de l'art* : *cloud computing*, conteneurisation, architecture des applications web modernes et principales menaces de sécurité (OWASP Top 10).
- Le **chapitre 2** formalise l'*analyse et la spécification des besoins* : acteurs, cas d'utilisation, besoins fonctionnels et non fonctionnels.
- Le **chapitre 3** présente la *conception* de la solution : architecture globale, infrastructure AWS, architecture Docker, modèle de données et conception de la sécurité **en profondeur**.
- Le **chapitre 4** détaille la *réalisation et le déploiement* : choix technologiques, implémentation, construction des images Docker, mise en place de l'instance EC2, configuration de Route 53, du proxy Nginx et des certificats Let's Encrypt.
- Le **chapitre 5** expose la phase de *tests fonctionnels et d'analyse de sécurité* : scans Nmap, audit Nikto, évaluation TLS et synthèse des recommandations.

Une *conclusion générale* dresse le bilan du projet et propose des pistes d'évolution.

5. Approche méthodologique

Le projet a été conduit selon une **démarche agile adaptée au contexte académique individuel**. Bien que les méthodologies agiles (Scrum, Kanban) soient traditionnellement conçues pour des équipes, leurs principes fondamentaux — itérations courtes, livraison incrémentale et amélioration continue — ont structuré l'ensemble du travail.

Trois pratiques agiles ont été retenues et adaptées :

1. **Sprints de 2-3 semaines** : chaque sprint visait un livrable concret (chapitre, composant, déploiement, tests).
2. **Revues avec l'encadrant** : points réguliers pour valider les orientations et ajuster les priorités.
3. **Tableau Kanban personnel** : suivi visuel des tâches (À faire, En cours, Terminé) pour limiter le travail en cours et maintenir un rythme soutenable.

5.1. Organisation des sprints

La planification a été organisée en cinq sprints principaux :

Sprint	Durée	Phase	Objectifs	Livrables
1	2 sem.	Analyse	État de l'art, cadrage	Chapitres 1-2
2	3 sem.	Conception	Architecture, UML	Chapitre 3
3	3 sem.	Réalisation	Développement, conteneurisation	Chapitre 4
4	2 sem.	Déploiement	AWS, HTTPS, CI/CD	Chapitre 4
5	2 sem.	Validation	Tests, audit sécurité	Chapitre 5

Tableau 1. – Organisation des sprints du projet WebCyber.

Cette organisation a favorisé une progression mesurable avec des revues régulières et des ajustements en continu.

Chapitre 1

1 CONTEXTE GÉNÉRAL ET ÉTAT DE L'ART

1. 1 Introduction

Ce premier chapitre situe le projet dans son contexte académique et technologique. Il rappelle d'abord le cadre institutionnel (l'iTeam University), puis présente les fondements théoriques mobilisés : *cloud computing*, conteneurisation, architecture des applications web modernes et grands principes de cybersécurité. Cette base conceptuelle permet de motiver, dans les chapitres suivants, les choix de conception et d'implémentation retenus.

1. 2 Cadre institutionnel

L'**iTeam University** est un établissement d'enseignement supérieur mauritanien qui forme depuis plusieurs décennies des cadres dans les domaines de la gestion, de la comptabilité et, plus récemment, du développement informatique. Le présent travail s'inscrit dans le parcours de **Licence en Développement Informatique** et constitue le **Projet de Fin d'Études** (PFE) clôturant ce cursus.

L'objectif pédagogique est double : (i) mettre en pratique de manière intégrée les compétences acquises durant la formation (programmation, bases de données, réseaux, systèmes, sécurité) et (ii) confronter l'étudiant à un scénario réaliste de cycle complet — de la spécification jusqu'au déploiement et à l'audit de sécurité.

1. 3 Cloud computing

1. 3.1 Définition

Le **cloud computing** désigne la mise à disposition à la demande, via Internet, de ressources informatiques (calcul, stockage, réseau, bases de données, services applicatifs). Le NIST en donne une définition de référence reposant sur cinq caractéristiques essentielles [1] :

- libre-service à la demande (*on-demand self-service*);
- accès large via le réseau (*broad network access*);
- mutualisation des ressources (*resource pooling*);
- élasticité rapide (*rapid elasticity*);
- facturation à l'usage (*measured service*).

1. 3.2 Modèles de service

Trois modèles de service principaux sont communément distingués :

- **IaaS** (*Infrastructure as a Service*) : le fournisseur expose des ressources virtualisées brutes (machines, réseaux, disques). Le client gère l'OS, le *middleware* et l'application. *Exemple* : AWS EC2.
- **PaaS** (*Platform as a Service*) : le fournisseur gère l'infrastructure et l'environnement d'exécution. Le client fournit le code applicatif. *Exemple* : AWS Elastic Beanstalk, Heroku.
- **SaaS** (*Software as a Service*) : le client consomme directement une application complète accessible via le Web. *Exemple* : Gmail, Microsoft 365.

Pour ce projet, le modèle retenu est l'**IaaS**, car il offre le meilleur compromis entre maîtrise pédagogique des couches techniques (système, réseau, conteneurs, sécurité) et simplicité de mise en œuvre.

1. 3.3 Le fournisseur AWS

Amazon Web Services est le leader mondial du *cloud* public, avec plus d'une trentaine de *régions* (zones géographiques indépendantes) et une offre couvrant l'ensemble des modèles de service. Les briques utilisées dans ce projet sont :

- **EC2** (*Elastic Compute Cloud*) : instances de machines virtuelles facturées à l'heure ou à la seconde;
- **Elastic IP** : adresse IPv4 publique fixe, associable à une instance EC2;
- **Security Group** : pare-feu *stateful* fonctionnant au niveau de l'instance;
- **Route 53** : service DNS managé (zone hébergée et enregistrement de domaine).

1. 4 Conteneurisation

1. 4.1 Du *bare-metal* aux conteneurs

L'évolution des modèles d'exécution applicatifs peut se résumer en trois grandes étapes :

1. **Bare-metal** : un serveur physique = une application. Coût élevé, ressources sous-utilisées.
2. **Virtualisation** : un serveur physique exécute plusieurs machines virtuelles, chacune contenant son OS complet. Meilleure densité, mais surcharge importante (chaque VM démarre un noyau).
3. **Conteneurs** : plusieurs processus isolés partagent le *noyau* de l'hôte, en utilisant les *namespaces* et les *cgroups* Linux pour cloisonner les ressources et la visibilité. Démarrage en quelques centaines de millisecondes, empreinte mémoire réduite.

1. 4.2 Docker et Docker Compose

Docker est aujourd'hui l'outil de référence pour la création et l'exécution de conteneurs. Il introduit deux concepts fondamentaux :

- l'**image** : un *instantané* immuable contenant l'ensemble des fichiers nécessaires à l'exécution d'une application (binaires, bibliothèques, configuration);
- le **conteneur** : une instance d'exécution d'une image, avec son propre espace de noms (réseau, processus, fichiers).

Docker Compose est un outil d'*orchestration locale* qui permet, à partir d'un unique fichier déclaratif YAML (`docker-compose.yml`), de décrire un ensemble de services, leurs relations, leurs réseaux et leurs volumes. Une seule commande suffit ensuite à démarrer ou arrêter l'ensemble de la pile applicative.

1. 4.3 Bénéfices pour ce projet

- **Reproductibilité** : le même fichier `docker-compose.yml` fonctionne à l'identique sur le poste de développement et sur l'instance EC2.
- **Isolation** : chaque service tourne dans son propre conteneur, avec ses dépendances précises.
- **Sécurité** : le réseau interne Docker permet de n'exposer que le proxy Nginx, en gardant l'application Flask et la base PostgreSQL invisibles depuis Internet.
- **Portabilité** : la migration vers un autre fournisseur *cloud* ne nécessite que de relancer la même pile sur une nouvelle machine.

1. 5 Architecture des applications web modernes

1. 5.1 Le modèle 3-tiers

Une application web suit classiquement un découpage en trois couches :

- **Présentation** (*front-end*) : navigateur du client, qui interprète HTML, CSS et JavaScript;
- **Logique métier** (*back-end*) : serveur applicatif qui traite les requêtes, exécute la logique et rend des réponses;
- **Données** : base de données qui assure la persistance.

1. 5.2 Le rôle du proxy inverse

Un **proxy inverse** est un serveur intermédiaire qui se place entre les clients et l'application. Il assure plusieurs fonctions essentielles :

- **Terminaison TLS** : il déchiffre le trafic HTTPS, ce qui évite à l'application de gérer elle-même les certificats;
- **Routage** : il dirige les requêtes vers le bon *back-end*;
- **Mise en cache** et compression;
- **Sécurité périmétrique** : ajout d'en-têtes, limitation de débit, filtrage.

Pour ce projet, le proxy choisi est **Nginx**, en raison de sa légèreté, de sa robustesse et de la richesse de sa documentation.

1. 6 Cybersécurité applicative

1. 6.1 Le Top 10 OWASP

L'**Open Web Application Security Project** (OWASP) publie périodiquement la liste des dix risques applicatifs les plus critiques. La version 2021, qui sert de référence à ce projet, met notamment l'accent sur [2] :

1. **Broken Access Control** : contournement des contrôles d'accès, notamment via la manipulation d'identifiants (*IDOR*);
2. **Cryptographic Failures** : stockage ou transmission de données sensibles sans chiffrement adéquat;
3. **Injection** : injection SQL, NoSQL, OS command;
4. **Insecure Design** : absence de modélisation de la menace en amont du développement;
5. **Security Misconfiguration** : valeurs par défaut, ports inutiles, en-têtes manquants;
6. **Vulnerable and Outdated Components** : utilisation de bibliothèques non maintenues;
7. **Identification and Authentication Failures**;
8. **Software and Data Integrity Failures**;
9. **Security Logging and Monitoring Failures**;
10. **Server-Side Request Forgery (SSRF)**.

1. 6.2 La défense en profondeur

La **défense en profondeur** (*defense-in-depth*) consiste à empiler plusieurs barrières de sécurité indépendantes, de telle sorte que la compromission d'une couche ne suffise pas à compromettre le système entier. Ce principe, hérité du domaine militaire, est particulièrement pertinent pour les applications hébergées dans le *cloud*, où les surfaces d'attaque sont multiples (réseau, système, conteneurs, code, données).

Le Chapitre @ montrera comment ce principe se traduit concrètement dans WebCyber par six couches de sécurité empilées.

1. 6.3 TLS et HTTPS

Transport Layer Security (TLS) est le protocole standard pour chiffrer les communications sur Internet. **HTTPS** désigne simplement HTTP encapsulé dans TLS. Trois propriétés sont garanties :

- **Confidentialité** : le contenu des échanges ne peut être lu par un tiers;
- **Intégrité** : le contenu ne peut être modifié sans détection;
- **Authenticité** : le client peut vérifier l'identité du serveur grâce à un certificat signé par une autorité de confiance.

Let's Encrypt est une autorité de certification gratuite fondée en 2014 par l'*Internet Security Research Group*, qui émet des certificats TLS valides 90 jours, renouvelables automatiquement via le protocole ACME.

1. 7 Conclusion du chapitre

Ce chapitre a présenté les concepts clés mobilisés tout au long du projet : modèle IaaS et services AWS, conteneurisation par Docker, architecture web 3-tiers, principes OWASP et défense en profondeur, ainsi que TLS et Let's Encrypt. Le chapitre suivant s'appuie sur ces fondations pour formaliser les besoins fonctionnels et non fonctionnels de l'application WebCyber.

Chapitre 2

2 ANALYSE ET SPÉCIFICATION DES BESOINS

2. 1 Introduction

Ce chapitre formalise ce que l'application doit faire et dans quelles conditions. Il présente successivement les acteurs, les besoins fonctionnels (regroupés en cas d'utilisation), les user stories agiles, le product backlog, puis les besoins non fonctionnels (sécurité, performance, déploiement, maintenabilité).

2. 2 Description générale du projet

WebCyber est une application web sécurisée de prise de notes personnelle. Chaque utilisateur, après s'être inscrit puis authentifié, peut créer, lire, modifier, archiver, supprimer (corbeille avec restauration) et rechercher des notes textuelles qui lui sont propres. Aucun utilisateur ne peut accéder aux notes d'un autre, même en manipulant directement les URL ou les paramètres de requête.

L'application n'est donc pas une innovation fonctionnelle en elle-même — des solutions comparables (Google Keep, Microsoft OneNote, Apple Notes) existent depuis longtemps. Le choix d'un domaine fonctionnel familier est volontaire : il permet de **concentrer l'effort sur la qualité de la conception, du déploiement et de la sécurité** plutôt que sur la nouveauté de la fonctionnalité.

2. 3 Identification des acteurs

Deux acteurs interagissent avec le système :

- **Visiteur** : tout internaute non authentifié. Il peut accéder uniquement aux pages publiques (page d'accueil, formulaire de connexion, formulaire d'inscription).
- **Utilisateur authentifié** : un visiteur qui s'est connecté avec succès. Il accède à son espace personnel et y gère ses notes.

L'application n'embarque pas d'acteur *Administrateur* : la gestion des utilisateurs (création, suppression d'un compte) reste manuelle, par accès direct à la base de données. Cette simplification est cohérente avec le périmètre académique.

2. 4 Besoins fonctionnels

2. 4.1 Liste des besoins

ID	Besoin	Description
BF-01	S'inscrire	Le visiteur saisit un nom d'utilisateur, un email et un mot de passe; le système crée un compte.
BF-02	Se connecter	L'utilisateur saisit ses identifiants; le système vérifie le hachage et ouvre une session.
BF-03	Se déconnecter	La session est détruite côté serveur; l'utilisateur revient à la page de connexion.
BF-04	Créer une note	L'utilisateur saisit un titre et un contenu; la note est sauvegardée et associée à son compte.
BF-05	Consulter ses notes	Affichage paginé des notes de l'utilisateur, par date de modification décroissante.
BF-06	Voir une note en détail	Affichage du titre et du contenu complet d'une note.
BF-07	Modifier une note	L'utilisateur édite le titre et/ou le contenu; la date de modification est mise à jour.
BF-08	Archiver / désarchiver	La note est marquée <code>is_archived</code> ; elle disparaît de la liste principale et apparaît dans l'archive.
BF-09	Mettre à la corbeille	La note est marquée <code>is_trashed</code> ; elle est cachée mais peut être restaurée.
BF-10	Restaurer depuis la corbeille	Le drapeau <code>is_trashed</code> est levé; la note revient dans la liste.
BF-11	Supprimer définitivement	Suppression DELETE en base, irrécupérable.
BF-12	Rechercher une note	Filtre par titre ou contenu; résultats limités au compte courant.
BF-13	Isolation par utilisateur	Toute lecture, écriture ou suppression filtre par <code>user_id</code> de la session courante.

Tableau 2. – Tableau des besoins fonctionnels.

2. 4.2 Diagramme de cas d'utilisation

La Figure @ synthétise les cas d'utilisation et leurs liens avec les acteurs. Les relations « *include* » indiquent que les opérations sur les notes nécessitent une authentification préalable (BF-02).

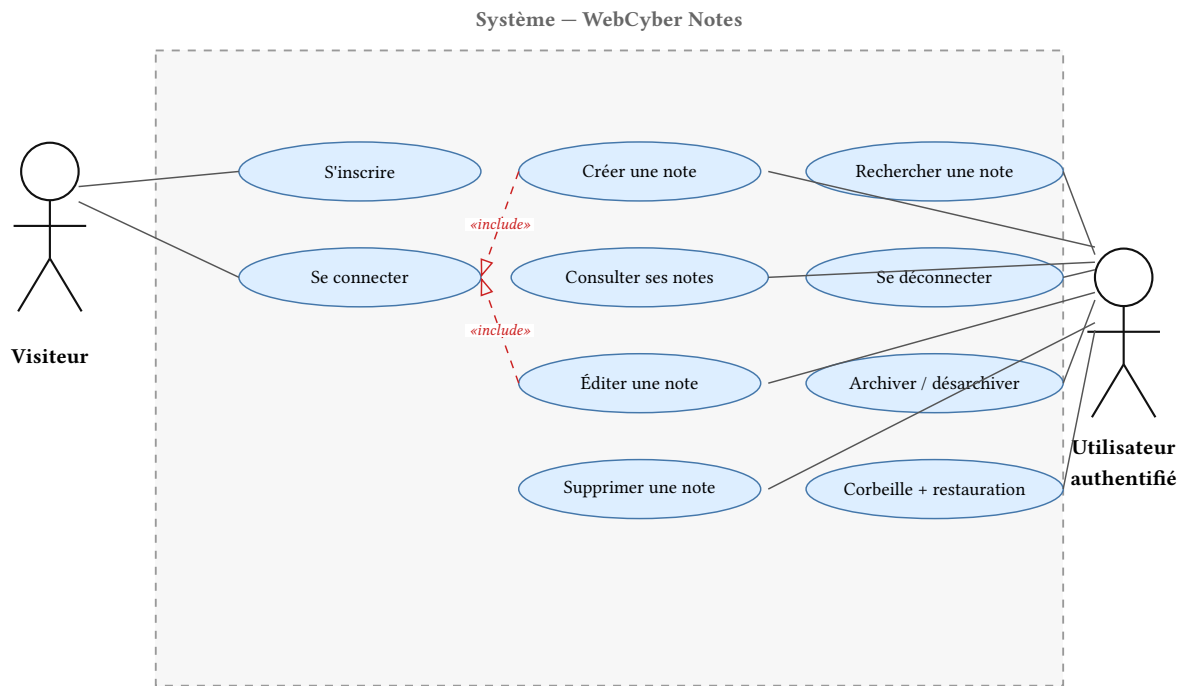


Figure 1. – Diagramme de cas d'utilisation de WebCyber.

2. 4.3 User Stories

Dans le cadre de la démarche agile adoptée (cf. introduction), les besoins fonctionnels ont été reformulés sous forme de **user stories** selon le format standard :

En tant que [acteur], je veux [action] afin de [bénéfice].

Chaque story est accompagnée de **critères d'acceptation** mesurables qui définissent les conditions de validation. Le Tableau @ ci-dessous présente l'ensemble des stories du projet.

ID	Acteur	Story	Critères d'acceptation
US-01	Visiteur	Créer un compte avec username, email et mot de passe	Username unique (3-80 car.), email valide et unique, mot de passe ≥ 8 car.
US-02	Visiteur	Me connecter avec mes identifiants	Session créée côté serveur, cookie sécurisé, redirection vers /notes
US-03	Utilisateur	Créer une note avec titre et contenu	Titre obligatoire (max 200 car.), note associée à mon user_id, horodatage automatique
US-04	Utilisateur	Consulter la liste de mes notes	Pagination, ordre antéchronologique, notes non archivées et non supprimées
US-05	Utilisateur	Voir le détail d'une note	Titre et contenu complets affichés, propriétaire uniquement
US-06	Utilisateur	Modifier le titre ou le contenu d'une note	updated_at actualisé, propriétaire uniquement, validation des champs
US-07	Utilisateur	Archiver une note	Note masquée de la liste principale, accessible dans l'archive, réversible
US-08	Utilisateur	Supprimer une note (corbeille)	Note masquée, apparaît dans la corbeille, restaurable
US-09	Utilisateur	Restaurer une note depuis la corbeille	Note réapparaît dans la liste principale, is_trashed = false
US-10	Utilisateur	Supprimer définitivement une note	Suppression DELETE en base, irréversible
US-11	Utilisateur	Rechercher parmi mes notes	Filtre par titre ou contenu, résultats limités à mon compte
US-12	Utilisateur	Être le seul à accéder à mes notes	Toute requête filtrée par user_id, accès direct $\rightarrow$ 404

Tableau 3. – User Stories du projet WebCyber.

2. 4.4 Product Backlog

Le **Product Backlog** (Tableau @) priorise les user stories selon leur valeur métier et leurs dépendances techniques. Les stories critiques (sécurité, isolation) ont été traitées dès les premiers sprints, conformément au principe **Security by Design**. Les stories de confort (archive, recherche) ont été planifiées dans les sprints ultérieurs.

ID	Story	Priorité	Sprint	Statut
US-01	Inscription	Haute	3	Terminé
US-02	Connexion	Haute	3	Terminé
US-12	Isolation données	Critique	3-5	Terminé
US-03	Créer une note	Haute	3	Terminé
US-04	Lister les notes	Haute	3	Terminé
US-05	Voir une note	Haute	3	Terminé
US-06	Modifier une note	Moyenne	4	Terminé
US-07	Archiver une note	Basse	4	Terminé
US-08	Supprimer (corbeille)	Moyenne	4	Terminé
US-09	Restaurer une note	Moyenne	4	Terminé
US-10	Suppression définitive	Basse	4	Terminé
US-11	Rechercher	Basse	4	Terminé

Tableau 4. – Product Backlog du projet WebCyber.

La priorisation suit une logique **MoSCoW** adaptée :

- **Critique** : indispensable à la sécurité ou au fonctionnement minimal.
- **Haute** : nécessaire pour la démonstration du produit.
- **Moyenne** : améliore significativement l'expérience utilisateur.
- **Basse** : fonctionnalité de confort, traitable en fin de projet.

2. 5 Besoins non fonctionnels

Les besoins non fonctionnels traduisent les *qualités attendues* du système, indépendamment des fonctionnalités elles-mêmes.

2. 5.1 Sécurité

- **Confidentialité** : tout le trafic entre le client et le serveur doit être chiffré (HTTPS, TLS 1.2 ou 1.3).
- **Authentification forte** : les mots de passe ne sont jamais stockés en clair; un algorithme de hachage à coût configurable (PBKDF2-SHA256) est utilisé.
- **Protection contre les vulnérabilités OWASP Top 10** : en particulier les injections SQL, le XSS, le CSRF et les contournements de contrôle d'accès.
- **Isolation réseau** : seuls les ports strictement nécessaires (22, 80, 443) sont exposés au public.
- **Isolation des données** : un utilisateur ne peut en aucun cas accéder aux notes d'un autre.

2. 5.2 Disponibilité et performance

- **Disponibilité** : le projet ne vise pas la haute disponibilité; une instance EC2 unique suffit. Le redémarrage automatique des conteneurs (restart: unless-stopped) couvre les pannes logicielles.
- **Performance** : la solution doit pouvoir servir une dizaine d'utilisateurs simultanés sur une instance t2.micro (1 vCPU, 1 Gio de RAM). Cela correspond largement aux besoins d'une démonstration ou d'un usage personnel.

2. 5.3 Déploiement et maintenabilité

- **Déploiement reproductible** : la solution doit pouvoir être redéployée *from scratch* à partir du dépôt Git, en moins de trente minutes.
- **Conteneurisation complète** : aucun composant n'est installé directement sur l'hôte; tout passe par Docker Compose.
- **Documentation** : l'ensemble des choix d'architecture, de configuration et de sécurité est documenté de manière à permettre à un autre étudiant ou ingénieur de prendre la suite.
- **Renouvellement automatisé** des certificats TLS via Certbot (cron quotidien).

2. 5.4 Ergonomie

- Interface web responsive (mobile et desktop), basée sur **Tailwind CSS** pour un rendu moderne et cohérent.
- Navigation simple : barre latérale pour basculer entre la liste, l'archive et la corbeille; palette de commandes pour un accès rapide.

2. 5.5 Périmètre exclu

Pour rester fidèle au cadre académique du PFE, certains éléments classiques d'une production réelle sont **volontairement exclus** :

- Orchestration Kubernetes ou multi-instance (*auto-scaling*);
- Pipeline CI/CD complet (GitHub Actions + déploiement automatique);
- *Web Application Firewall*, services de protection AWS avancés (WAF, GuardDuty);
- Centralisation des journaux (CloudWatch, ELK);
- Monitoring applicatif (Prometheus, Grafana).

Ces éléments sont mentionnés dans la **conclusion générale** comme perspectives d'évolution.

2. 6 Conclusion du chapitre

L'analyse des besoins fait ressortir un système fonctionnellement simple mais soumis à des exigences non fonctionnelles *nombreuses* (sécurité, déploiement reproductible, isolation, HTTPS). La formalisation en user stories et le product backlog ont structuré le développement en sprints cohérents, chaque itération apportant un incrément fonctionnel validable. Le chapitre suivant traduit ces exigences en une architecture concrète : choix de l'infrastructure AWS, organisation des conteneurs Docker, modèle de données, conception de la sécurité en profondeur.

Chapitre 3

3 CONCEPTION

3. 1 Introduction

Ce chapitre traduit les besoins du chapitre précédent en une **architecture technique** précise. Il présente d'abord l'architecture globale du système, puis l'infrastructure AWS et l'architecture Docker, le modèle de données, plusieurs scénarios d'interaction (diagrammes de séquence) et enfin la conception de la sécurité en profondeur.

3. 2 Architecture globale

L'architecture cible peut être résumée par la Figure @. Le navigateur du client résout d'abord le nom de domaine `webcyber.app` via Route 53, qui retourne l'**Elastic IP** de l'instance EC2. La requête HTTPS atteint ensuite l'instance, où elle traverse le **Security Group** avant d'être servie par le conteneur Nginx, qui terminera TLS et *proxifiera* la requête vers l'application Flask.

Figure 01 — Architecture globale

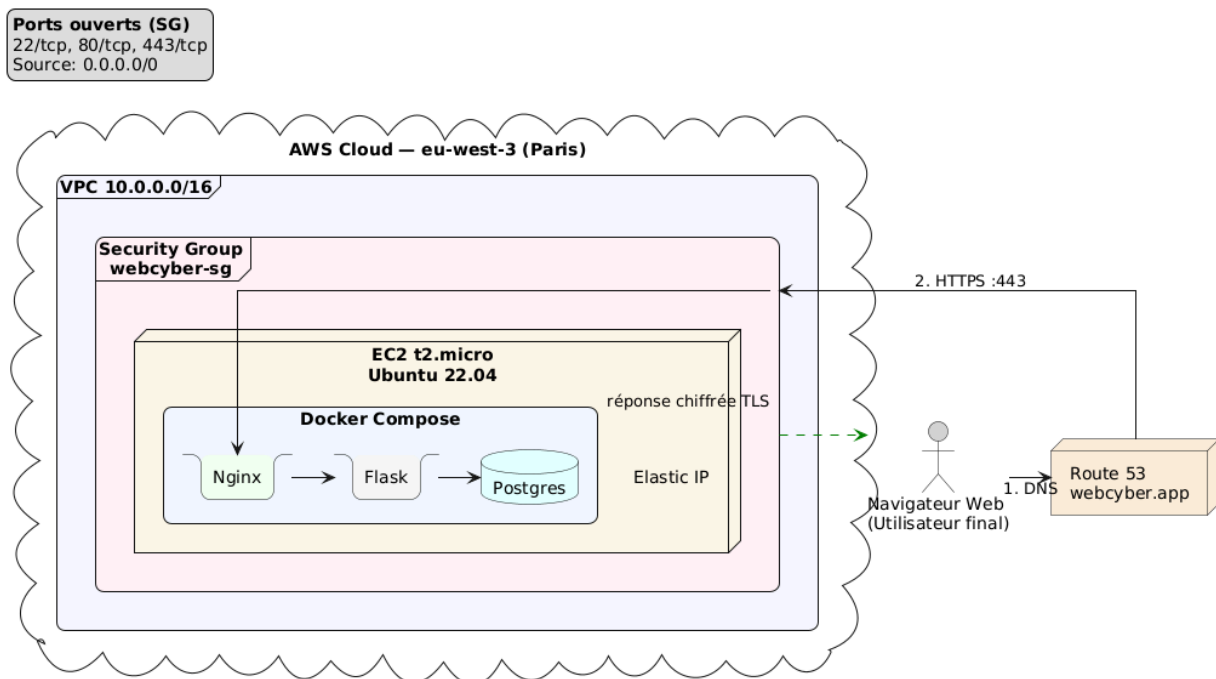


Figure 1. – Architecture globale - vue d'ensemble.

3. 2.1 Composants et responsabilités

Composant	Responsabilité
Route 53	Résolution DNS : <code>webcyber.app</code> → Elastic IP.
Elastic IP	Adresse IPv4 publique fixe associée à l'EC2.
EC2 (t2.micro)	Hôte Ubuntu 22.04 exécutant Docker.
Security Group	Pare-feu <i>stateful</i> au niveau de l'EC2.
Conteneur Nginx	Proxy inverse, terminaison TLS, en-têtes de sécurité.
Conteneur Flask	Logique applicative, authentification, CRUD.
Conteneur PostgreSQL	Persistance des données utilisateur.

Tableau 5. – Tableau des composants et responsabilités

3. 3 Infrastructure AWS

3. 3.1 Choix de la région

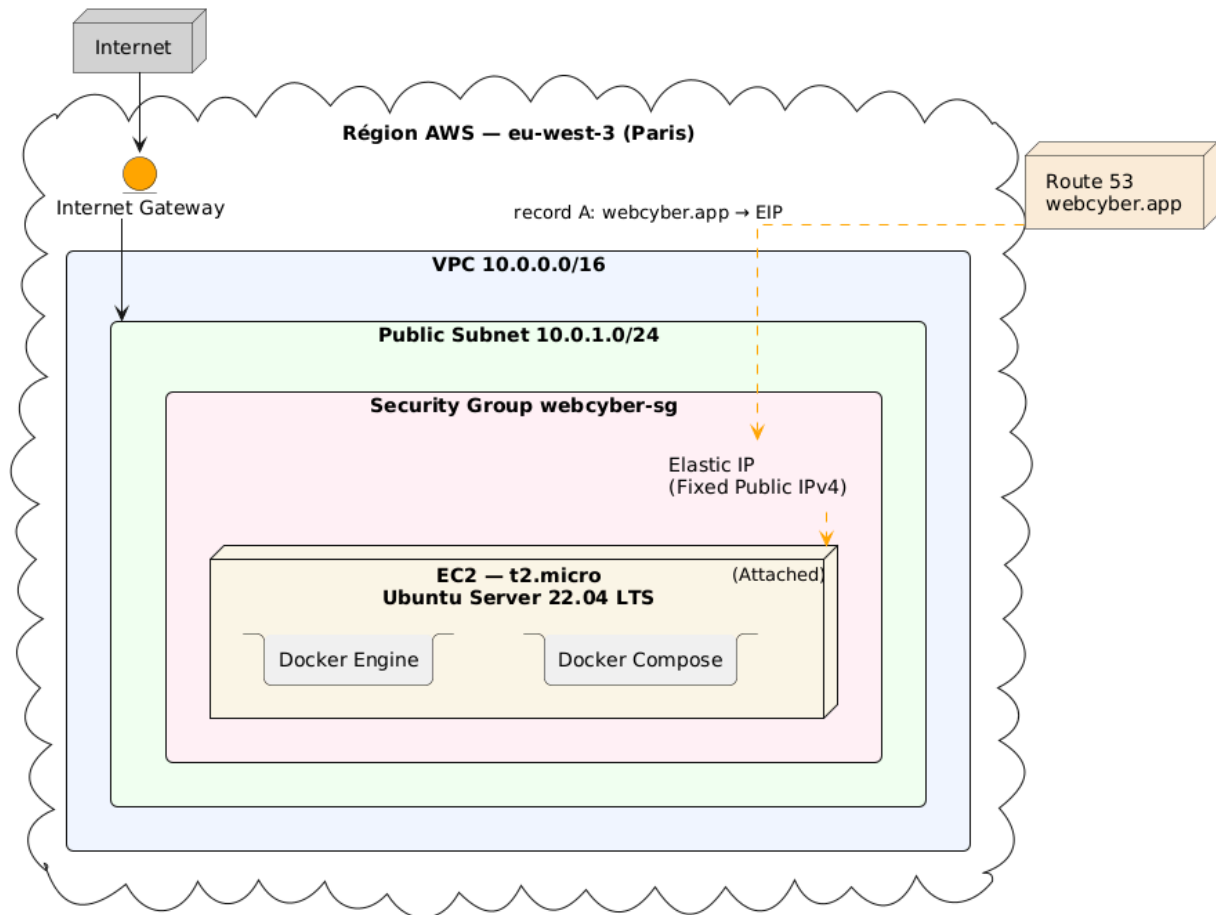
La région eu-west-3 (Paris) a été choisie pour sa proximité géographique (latence réduite vers la Mauritanie et l'Europe francophone) et pour des considérations de *résidence des données* (stockage en Europe).

3. 3.2 Composants AWS retenus

La Figure @ détaille l'organisation interne au *cloud*. Les choix retenus privilégient la **simplicité** et la **lisibilité** pédagogique :

- utilisation du **VPC par défaut** (CIDR 172.31.0.0/16) sans VPC personnalisé;
- un **sous-réseau public** unique;
- un **Security Group dédié** (`webcyber-sg`);
- une seule **instance EC2** t2.micro;
- une **Elastic IP** associée;
- une **zone hébergée Route 53** pour le domaine `webcyber.app`.

Figure 02 — Infrastructure AWS détaillée



Règles entrantes (SG)		
Port	Proto	Source
22	TCP	0.0.0.0/0
80	TCP	0.0.0.0/0
443	TCP	0.0.0.0/0

Figure 2. – Infrastructure AWS détaillée.

3. 3.3 Règles du Security Group

Le pare-feu réseau du périmètre est un **Security Group** *stateful* : seul le trafic entrant explicitement autorisé est accepté, et les réponses correspondantes sont automatiquement autorisées en sortie.

Type	Proto- cole	Port	Source	Justification
SSH	TCP	22	0.0.0.0/0	Administration distante
HTTP	TCP	80	0.0.0.0/0	Redirection vers HTTPS
HTTPS	TCP	443	0.0.0.0/0	Trafic web chiffré

Tableau 6. – Règles entrantes du Security Group `webcyber-sg`.

Aucun autre port n'est ouvert. Les ports applicatifs (5000 pour Flask, 5432 pour PostgreSQL) restent confinés au réseau Docker interne et ne sont donc pas accessibles depuis Internet.

3. 3.4 Estimation des coûts

Ressource	Coût
EC2 t2.micro	0,00
«EBS 20 Gio gp3»	0,00
Elastic IP (associée)	0,00
«Route 53 zone hébergée»	0,50
Trafic sortant (faible)	~ 0,00
Total estimé	~ 0,50/mois

Tableau 7. – Estimation mensuelle (sous _free tier_ AWS).

3. 4 Architecture Docker

3. 4.1 Vue d'ensemble

Trois conteneurs cohabitent sur l'instance, orchestrés par Docker Compose et reliés par un réseau *bridge* privé (*webcyber-net*). La Figure @ montre cette organisation, ainsi que les volumes persistants utilisés.

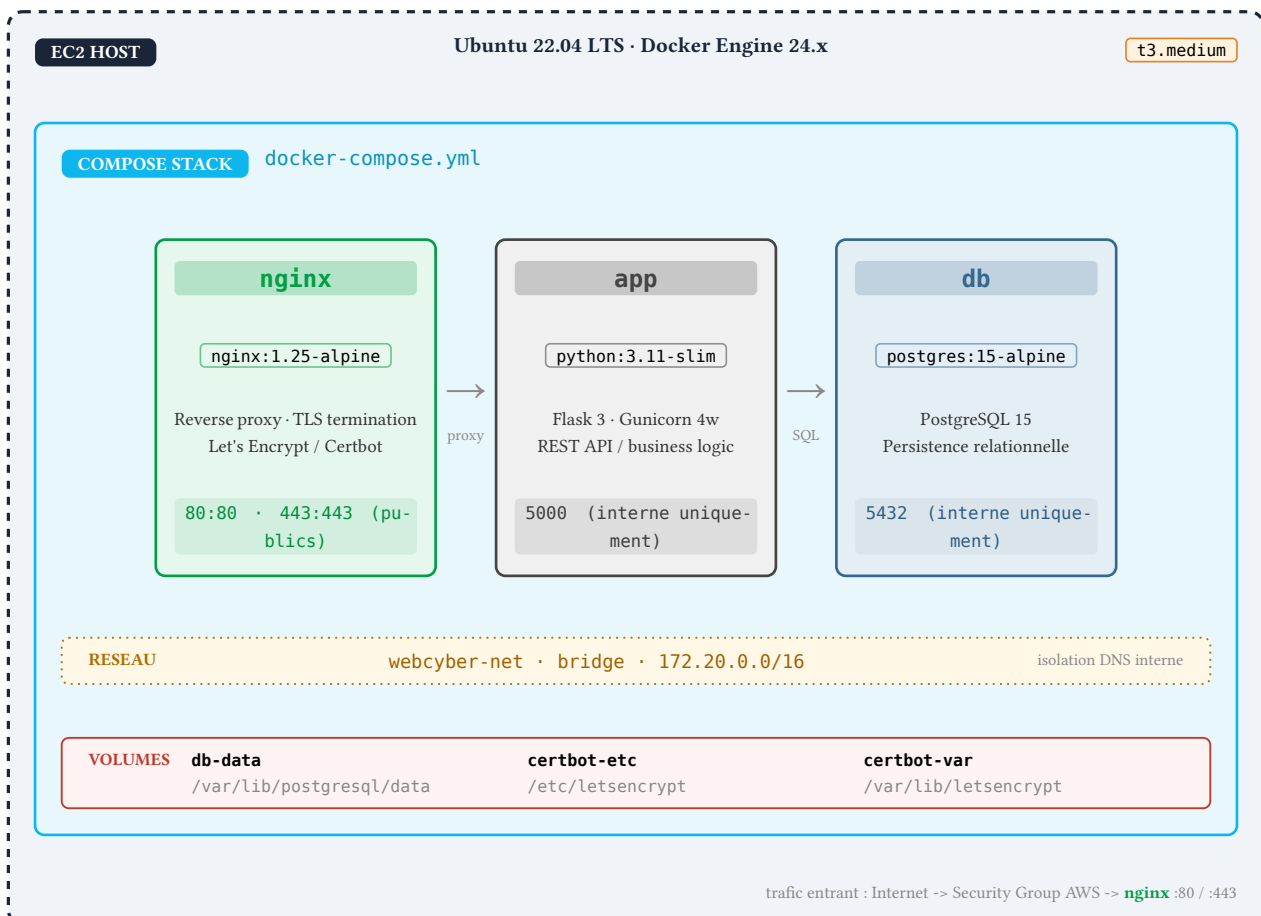


Figure 2. – Architecture Docker - trois conteneurs sur réseau _bridge_.

3. 4.2 Spécifications des conteneurs

Service	Image	Ports	Rôle
nginx	nginx:1.25-alpine	80, 443 (publiés)	Proxy inverse, TLS, en-têtes de sécurité
app	python:3.11-slim (build)	5000 (interne)	Application Flask + Gunicorn
db	postgres:15-alpine	5432 (interne)	Base de données PostgreSQL

Tableau 8. – Spécifications des conteneurs

Le **principe d'exposition minimale** est appliqué : seul le proxy publie ses ports vers l'hôte. Les services app et db ne sont accessibles qu'*intra-Docker*, par leur nom de service (résolution DNS embarquée par Docker).

3. 4.3 Volumes persistants

- db-data : données PostgreSQL (table users, table notes, indexes, WAL).
- certbot-etc : certificats Let's Encrypt et clés privées.
- certbot-var : état interne de Certbot (compte ACME, *lock files*).

3. 4.4 Estimation des ressources

Sur t2.micro (1 Gio RAM), l'occupation mémoire prévisionnelle est confortable :

Conteneur	Mémoire estimée
Nginx	10-30 Mio
Flask + 2 <i>workers</i> Gunicorn	100-200 Mio
PostgreSQL	50-100 Mio
Surcharge Docker	100 Mio
Total	300-430 Mio

Tableau 9. – Empreinte mémoire estimée des conteneurs.

3. 5 Modèle de données

Le modèle de données est volontairement réduit à deux entités, comme le montre la Figure @.

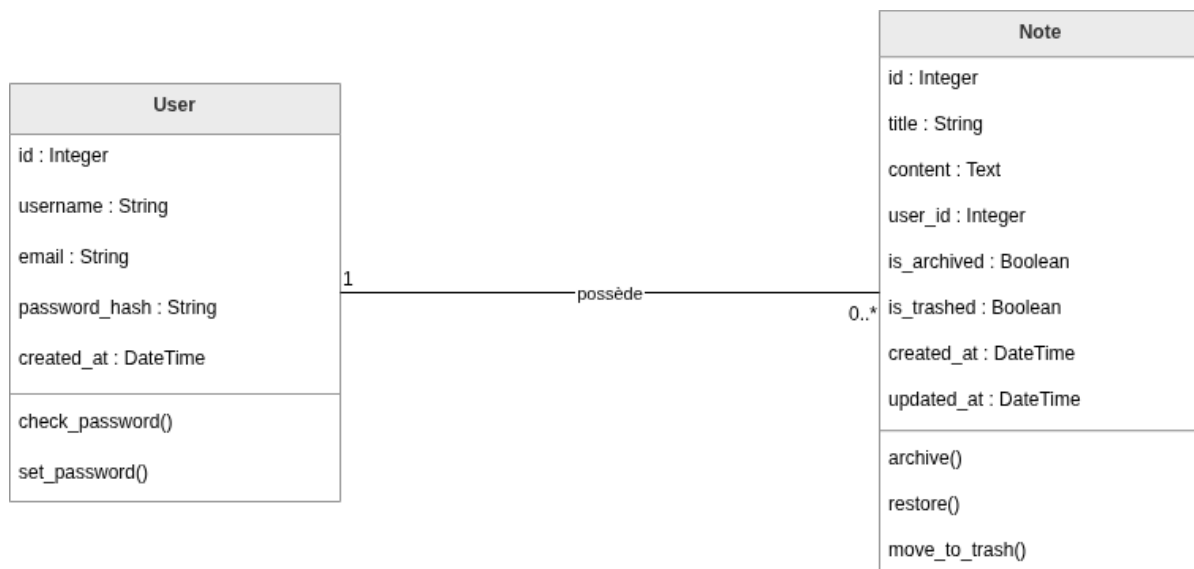


Figure 3. – Diagramme de classes (modèle de données).

3. 5.1 Description

- Un **User** possède de zéro à n **Notes**.
- Une **Note** appartient à exactement un **User**, via la clé étrangère `user_id`.
- Les drapeaux `is_archived` et `is_trashed` permettent une suppression *douce* (corbeille) et une archive sans déplacer ni dupliquer la donnée.

3. 5.2 Considérations de sécurité dans le modèle

- Le champ `password_hash` contient uniquement le *hachage* du mot de passe, jamais sa valeur en clair.
- Les contraintes `UNIQUE` sur `username` et `email` préviennent les doublons et certaines attaques d'énumération.
- Toutes les requêtes `SELECT`, `UPDATE` ou `DELETE` sur la table `notes` **doivent** comporter le filtre `WHERE user_id = current_user.id`.

3. 6 Diagrammes de séquence

3. 6.1 Authentification

Le scénario d'authentification met en jeu cinq acteurs : l'utilisateur, son navigateur, Nginx, Flask et PostgreSQL. La Figure @ en détaille le déroulement, depuis la saisie des identifiants jusqu'à la création de la session côté serveur et au positionnement du *cookie* dans le navigateur.

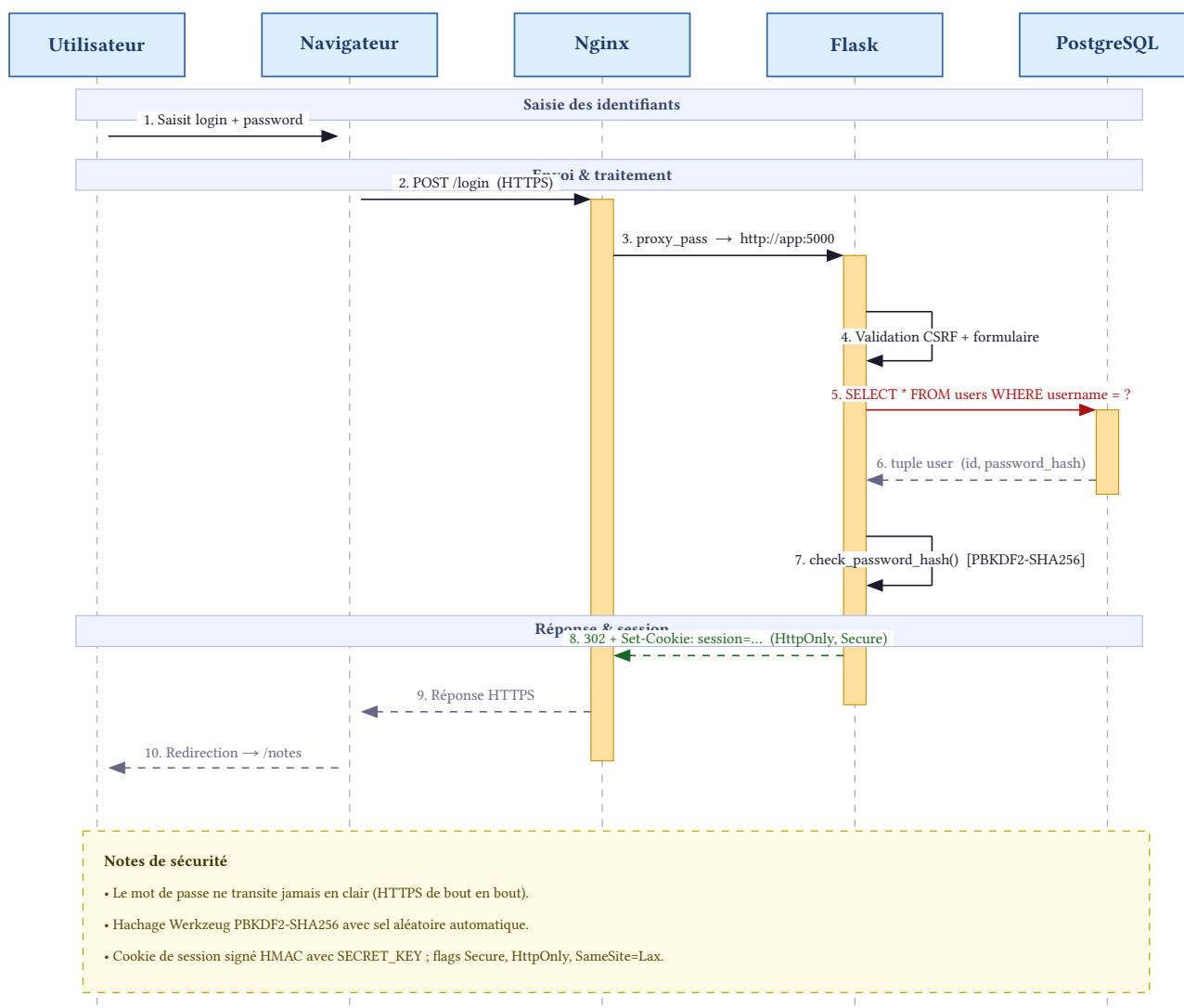


Figure 3. – Diagramme de séquence - authentification.

Trois éléments de sécurité sont à noter :

1. le mot de passe ne quitte jamais le tunnel TLS;
2. la comparaison côté serveur passe par `check_password_hash`, qui exécute une comparaison *constant-time*;
3. le *cookie* de session est signé (HMAC), et porte les attributs Secure, HttpOnly et SameSite=Lax.

3. 6.2 Création d'une note

Le scénario de création d'une note (Figure @) montre en particulier le contrôle de session (@login_required), la vérification du jeton CSRF, puis l'insertion en base avec association à l'utilisateur courant.

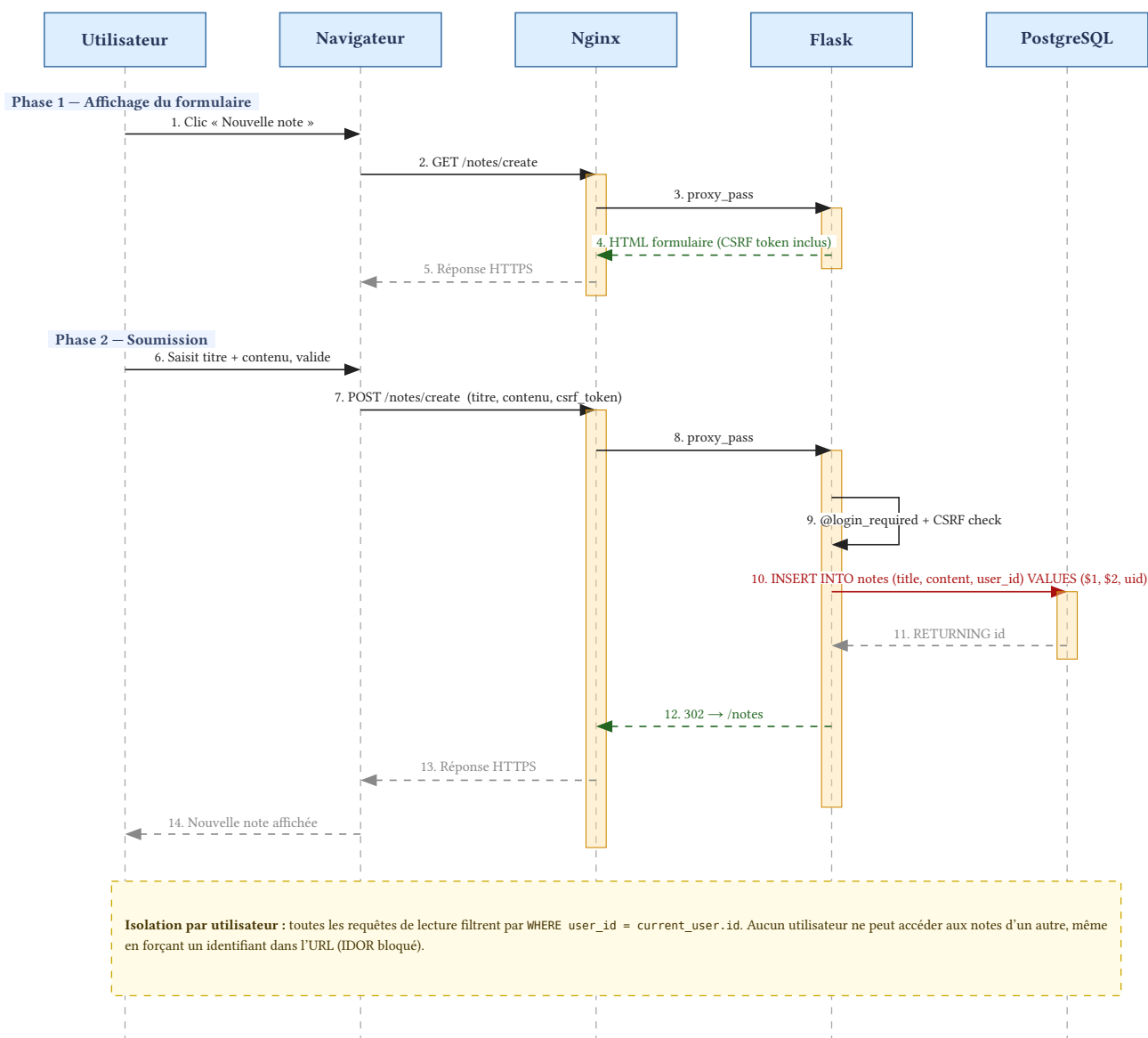


Figure 4. – Diagramme de séquence - création d'une note.

3. 7 Conception de la sécurité - défense en profondeur

La sécurité du système ne repose pas sur une unique mesure, mais sur **six couches empilées**, chacune apportant une garantie spécifique. La Figure @ en donne une représentation synthétique.

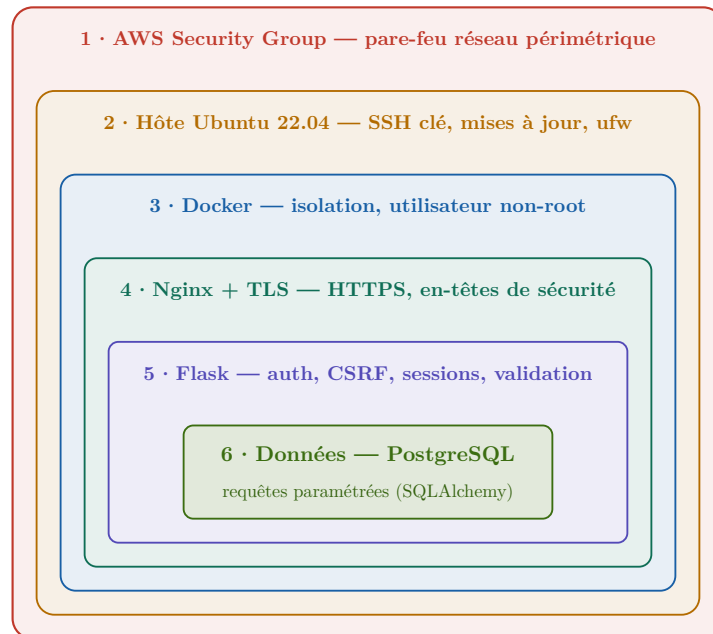


Figure 5. – Les six couches de sécurité (_defense-in-depth_).

3. 7.1 Couche 1 - Security Group

Le Security Group AWS est un pare-feu *stateful* qui constitue la **première ligne de défense** : il bloque tout port non explicitement autorisé. Sa configuration a été détaillée plus haut.

3. 7.2 Couche 2 - Système hôte

Sur l'instance EC2 :

- PermitRootLogin no : interdiction du login SSH root;
- authentification SSH par clé en mode primaire;
- unattended-upgrades pour appliquer les mises à jour de sécurité automatiquement;
- utilisateur applicatif ubuntu non-root.

3. 7.3 Couche 3 - Isolation Docker

- Réseau *bridge* privé webcyber-net; les conteneurs app et db ne publient pas de port;
- Le **Dockerfile** de l'application crée un utilisateur non-privilégié appuser (UID 1000) et l'utilise pour l'exécution finale;
- Les volumes nommés isolent les données persistantes du système de fichiers de l'hôte.

3. 7.4 Couche 4 - Nginx + TLS

- TLS 1.2 et 1.3 uniquement, profil *Mozilla Intermediate*;
- certificat **Let's Encrypt** renouvelé automatiquement;
- redirection 80 → 443;
- en-têtes **HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy** injectés sur toutes les réponses;
- server_tokens off pour masquer la version de Nginx.

3. 7.5 Couche 5 - Application Flask

- Hachage des mots de passe via `generate_password_hash` (PBKDF2-SHA256, sel automatique);
- Session signée par `SECRET_KEY` aléatoire (32 octets);
- Protection **CSRF** sur tous les formulaires (Flask-WTF);
- Validation systématique des entrées par WTForms (longueur, type, présence);
- Décorateur `@login_required` sur toutes les routes privées;
- Filtre `user_id` sur 100% des requêtes *notes*.

3. 7.6 Couche 6 - Base de données

- Utilisateur `webcyber_user` non-superutilisateur;
- Mot de passe long et aléatoire (≥ 20 caractères) injecté via `.env`;
- Pas de port publié vers l'hôte;
- Requêtes paramétrées par SQLAlchemy (pas de concaténation de chaînes SQL).

3. 8 Conclusion du chapitre

La conception ainsi décrite est volontairement *minimale mais complète* : un seul serveur, trois conteneurs, deux entités, mais une sécurité empilée sur six niveaux. Le chapitre suivant présente la réalisation concrète de cette conception et les étapes de son déploiement.

Chapitre 4

4 RÉALISATION ET DÉPLOIEMENT

4. 1 Introduction

Ce chapitre décrit la réalisation concrète de l'application WebCyber : choix technologiques détaillés, construction des images Docker, configuration de l'instance EC2, mise en place du DNS via Route 53, configuration de Nginx avec HTTPS Let's Encrypt, et procédure de déploiement *from scratch*. Il se termine par le diagramme de déploiement UML qui synthétise l'ensemble.

4. 2 Environnement et outils

4. 2.1 Poste de développement

- OS : Linux (Ubuntu 24.04);
- IDE : Visual Studio Code;
- Outils en ligne de commande : git, docker, docker compose, terraform, tfsec, ssh, curl, dig;
- Gestionnaire de versions : Git, dépôt distant sur GitHub.

4. 2.2 Stack technologique retenue

Couche	Choix	Justification
Cloud / IaaS	AWS EC2	Standard du marché, <i>free tier</i> , riche écosystème.
Système	Ubuntu 24.04 LTS	LTS, large support Docker.
IaC	Terraform 1.12 + provider AWS 5.98	Provisionnement déclaratif et reproductible de l'infra.
Conteneurs	Docker + Compose	Reproductibilité, isolation.
Registre images	GitHub Container Registry (GHCR)	Distribution de l'image applicative.
CI/CD	GitHub Actions	Pipeline build / scan / déploiement automatisé.
Sécurité IaC	tfsec (Aqua)	Analyse statique du code Terraform (SAST infra).
Proxy / TLS	Nginx 1.25	Léger, robuste, terminaison TLS éprouvée.
Langage	Python 3.11	Productivité, écosystème mature.
Framework web	Flask 3	Léger, transparent, pédagogique.
WSGI	Gunicorn	WSGI de production, multi- <i>worker</i> .
ORM	SQLAlchemy 2	Empêche par construction l'injection SQL.
Auth	Flask-Login	Gestion de session simple et bien testée.
Formulaires	Flask-WTF	Validation + CSRF intégrée.
Hash mot de passe	Werkzeug PBKDF2-SHA256	Sel automatique, coût configurable.
Base de données	PostgreSQL 16	Robuste, indexation et JSON natifs.
TLS / CA	Let's Encrypt + Certbot	Certificats gratuits et automatiques.
Front-end	Tailwind CSS	Mise en page rapide et responsive.

Tableau 10. – Synthèse de la pile technologique.

4. 3 Implémentation de l'application Flask

4. 3.1 Organisation du code

Le code source de l'application est structuré selon le *pattern application factory* de Flask, qui sépare la création de l'application, les *blueprints* (groupes de routes) et les modèles de données. L'arborescence principale se compose des répertoires `routes/` (authentification et CRUD notes), `templates/` (vues Jinja2 avec échappement automatique) et `static/` (ressources front-end).

4. 3.2 Modèles SQLAlchemy

Deux modèles principaux structurent la persistance : `User` (gestion des comptes avec hachage PBKDF2-SHA256) et `Note` (CRUD complet avec isolation par `user_id`). Les champs `is_archived` et `is_trashed` implémentent une suppression douce sans duplication des données.

4. 3.3 Authentification et isolation

Le mécanisme d'isolation repose sur un filtre systématique `filter_by(user_id=current_user.id)` appliqué sur toutes les requêtes aux notes. Cette approche garantit qu'un utilisateur ne peut accéder aux notes d'un autre, même en manipulant directement les identifiants dans l'URL (parade contre l'**IDOR**). Le décorateur `@login_required` protège toutes les routes privées.

4. 4 Conteneurisation

4. 4.1 Dockerfile applicatif

L'image Docker de l'application applique trois bonnes pratiques de sécurité : (i) utilisation d'une image *slim* pour réduire la surface d'attaque, (ii) création d'un utilisateur non-root `appuser` (UID 1000) pour l'exécution, (iii) Gunicorn comme serveur WSGI de production avec deux *workers*.

4. 4.2 Fichier docker-compose.yml

L'orchestration définit trois services interconnectés via un réseau *bridge* privé `webcyber-net` :

- **Nginx** : expose les ports 80 et 443, monte les volumes de certificats Let's Encrypt;
- **App** : construit depuis le Dockerfile, expose le port 5000 en interne uniquement;
- **DB** : PostgreSQL 15 Alpine, données persistantes via le volume `db-data`.

Le principe d'exposition minimale est respecté : seul Nginx publie ses ports vers l'hôte. Les variables sensibles (mots de passe, clés) sont injectées via un fichier `.env` non versionné.

4. 5 Infrastructure as Code avec Terraform

4. 5.1 Motivation

Plutôt que de provisionner l'infrastructure manuellement via la console AWS, l'infrastructure est décrite **en code** avec Terraform. Cette approche *Infrastructure as Code (IaC)* apporte reproductibilité, versionnement Git, auditabilité via `terraform plan`, et analyse statique avec `tfsec` avant déploiement.

4. 5.2 Ressources provisionnées

Le module Terraform définit trois ressources principales :

1. Une *data source* AMI Ubuntu 24.04 LTS (dernière version publiée par Canonical);
2. Une instance EC2 t3.micro avec IMDSv2 obligatoire et installation automatisée de Docker via `remote-exec`;
3. Un Security Group limitant l'accès aux ports 22 (SSH), 80 (HTTP) et 443 (HTTPS).

L'association d'une Elastic IP existante garantit la stabilité de l'adresse publique, essentielle pour la cohérence des enregistrements DNS. Le cycle de provisionnement suit les commandes standards : `init`, `plan`, `apply`.

4. 6 Configuration DNS via Route 53

Sur Route 53, une zone hébergée publique est créée pour `webcyber.app` et deux enregistrements de type A sont ajoutés :

Nom	Type	Valeur	TTL
<code>webcyber.app</code>	A	Elastic IP	300
<code>www.webcyber.app</code>	A	Elastic IP	300

Tableau 11. – Enregistrements DNS de la zone hébergée `webcyber.app`.

Le TLD `.app` étant inscrit sur la liste *HSTS preload*, le navigateur refuse toute connexion non-HTTPS, rendant la mise en place de Let's Encrypt obligatoire.

4. 7 Configuration Nginx et HTTPS

4. 7.1 Configuration du proxy

Le proxy Nginx est configuré en deux blocs : un bloc HTTP (port 80) redirigeant systématiquement vers HTTPS, et un bloc HTTPS (port 443) assurant la terminaison TLS et le *proxy* vers l'application Flask.

Les en-têtes de sécurité suivants sont injectés sur toutes les réponses :

En-tête	Rôle
<code>Strict-Transport-Security</code>	Force le navigateur à utiliser HTTPS pendant un an (HSTS).
<code>X-Content-Type-Options</code>	Empêche le <i>MIME-sniffing</i> (nosniff).
<code>X-Frame-Options</code>	Bloque l'inclusion en <code><iframe></code> sauf en <i>same-origin</i> (anti-clickjacking).
<code>Referrer-Policy</code>	Limite la fuite d'URL via le <i>referer</i> .
<code>Content-Security-Policy</code>	Restreint les sources de scripts, styles, images, polices (défense contre XSS).
<code>server_tokens off</code>	Masque la version de Nginx dans les en-têtes (réduction de la divulgation d'information).

Tableau 12. – En-têtes de sécurité HTTP appliqués par Nginx.

4. 7.2 Durcissement des cookies de session

La configuration Flask durcit les attributs des cookies de session : `SESSION_COOKIE_SECURE` (HTTPS uniquement en production), `SESSION_COOKIE_HTTPONLY` (inaccessible au JavaScript) et `SESSION_COOKIE_SAMESITE=Lax` (protection CSRF de base). Ces attributs sont vérifiés au Chapitre @.

4. 7.3 Certificat Let's Encrypt

Le certificat est obtenu via Certbot en mode *webroot*, puis renouvelé automatiquement par une tâche cron quotidienne qui recharge la configuration Nginx après renouvellement.

4. 8 Pipeline CI/CD avec GitHub Actions

4. 8.1 Vue d'ensemble

À chaque *push* sur la branche main, un *workflow* GitHub Actions en trois *jobs* séquentiels est déclenché :

No	Job	Rôle
1	infra-scan	Scan tfsec sur le dossier terraform/ (gate sécurité IaC).
2	build-and-push	docker build + push vers GHCR avec tags latest et sha.
3	deploy	SSH vers l'EC2, docker compose pull && up, gestion du certificat Let's Encrypt au premier déploiement.

Tableau 13. – Étapes du pipeline GitHub Actions.

Le *job* infra-scan utilise tfsec avec l'option `soft_fail: false` : toute vulnérabilité de niveau *HIGH* ou *CRITICAL* bloque le pipeline avant le déploiement.

4. 8.2 Secrets GitHub utilisés

Nom du secret	Contenu
EC2_HOST	Adresse IP publique (Elastic IP) de l'instance.
EC2_USER	ubuntu (utilisateur SSH par défaut).
EC2_SSH_KEY	Contenu de la clé privée vockey.pem.
ENV_FILE	Contenu complet du fichier .env (clé secrète Flask, identifiants PostgreSQL, etc.).

Tableau 14. – Secrets configurés dans le dépôt GitHub.

4. 8.3 Bénéfices de l'approche

- **Boucle de retour rapide** : un *push* suffit pour livrer l'application en production en quelques minutes.
- **Séparation des responsabilités** : Terraform gère l'infrastructure, GitHub Actions gère l'application.
- **Sécurité par construction** : aucun *commit* contenant une mauvaise configuration Terraform ne peut atteindre la production grâce au *gate* tfsec.

- **Aucun secret dans le dépôt** : le fichier `.env` et la clé SSH sont stockés exclusivement dans les *GitHub Secrets*.

4.9 Vue d'ensemble du flux HTTPS

La Figure @ synthétise le trajet d'une requête HTTPS depuis le navigateur jusqu'à la base de données, en distinguant la portion chiffrée (client à Nginx) de la portion en clair sur le réseau Docker isolé (Nginx, Flask, PostgreSQL).

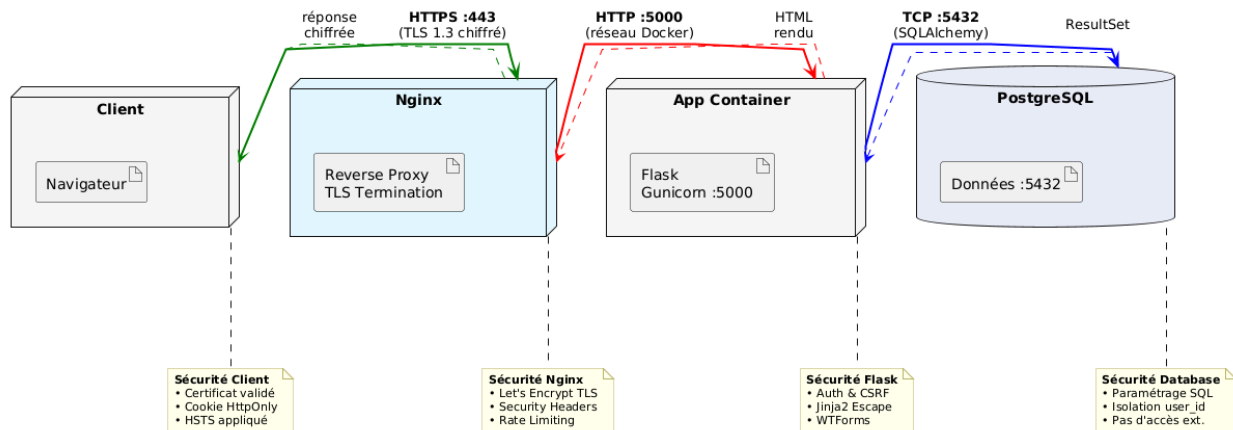


Figure 4. – Flux d'une requête HTTPS, des en-têtes au stockage.

4.9.1 Captures d'écran

Les figures suivantes illustrent l'interface de l'application déployée.



Figure 5. – Interface de connexion / inscription.



Figure 6. – Interface de gestion des notes.

4. 10 Diagramme de déploiement

Le diagramme de déploiement UML (Figure @) résume les artefacts physiques et logiques mis en jeu, ainsi que les protocoles de communication entre eux.

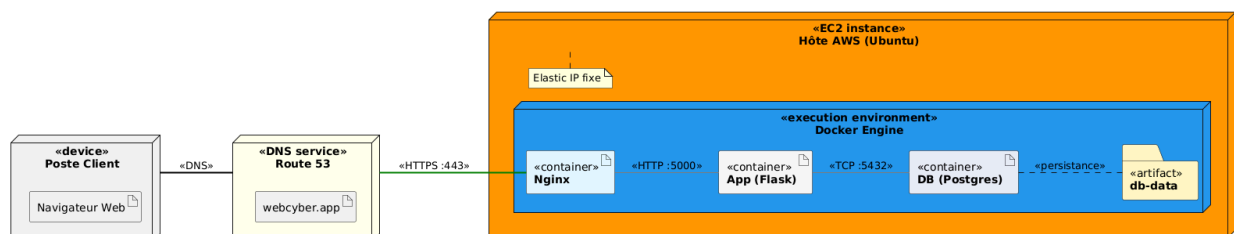


Figure 7. – Diagramme de déploiement UML.

4. 11 Conclusion du chapitre

La réalisation s'est appuyée sur des outils *standards de l'industrie* (Flask, PostgreSQL, Nginx, Docker, AWS, Let's Encrypt), ce qui a permis de concentrer l'effort sur l'**intégration sécurisée** plutôt que sur la maîtrise

de technologies exotiques. Le résultat : un déploiement reproductible, conteneurisé, accessible en HTTPS sur <https://webcyber.app>, et préparé pour la phase de tests et d'audit qui fait l'objet du Chapitre @.

Chapitre 5

5 TESTS ET ANALYSE DE SÉCURITÉ

5. 1 Introduction

Ce chapitre présente la dernière phase du projet : la **validation** fonctionnelle de l'application et l'**audit de sécurité** du déploiement. La méthodologie suit six phases complémentaires couvrant aussi bien le code applicatif que l'infrastructure : tests fonctionnels, analyse statique de l'IaC, vérification de l'infrastructure déployée, scan de ports, scan applicatif web, et évaluation de la configuration TLS.

5. 2 Stratégie de test

Phase	Type	Outils	Objectif
1	Fonctionnel	Navigateur, curl	Vérifier les cas d'utilisation
2	Analyse statique IaC	tfsec	Détecter les défauts de configuration Terraform <i>avant</i> déploiement
3	Infrastructure	docker, dig, curl	Confirmer le déploiement
4	Scan de ports	Nmap	Limiter la surface d'attaque
5	Scan applicatif web	Nikto, curl	Détecter les mauvaises configurations HTTP
6	Évaluation TLS	SSL Labs, testssl.sh	Mesurer la qualité du chiffrement

Tableau 15. – Phases de la campagne de validation.

5. 3 Tests fonctionnels

5. 3.1 Inscription et authentification

Les tests d'inscription et d'authentification couvrent à la fois les chemins nominaux (*happy path*) et les chemins d'erreur :

No	Cas	Entrée	Résultat attendu
1	Inscription valide	user/email/mdp corrects	Compte créé, redirection /login
2	Username existant	username déjà pris	Erreur, pas de doublon
3	Mot de passe court	< 8 caractères	Erreur de validation
4	Champs vides	blanc	Erreur de validation

Tableau 16. – Tests d'inscription.

No	Cas	Entrée	Résultat attendu
1	Identifiants valides	user/mdp corrects	Redirection /notes
2	Mauvais mot de passe	mdp erroné	Message d'erreur
3	Utilisateur inconnu	username inexistant	Message d'erreur
4	Accès direct sans auth	GET /notes	Redirection /login
5	Déconnexion	clic « se déconnecter »	Session détruite, retour /login

Tableau 17. – Tests d'authentification.

5. 3.2 Gestion des notes (CRUD)

No	Cas	Résultat attendu
1	Création de note	Note dans la liste
2	Détail d'une note	Titre + contenu complet affichés
3	Édition	Contenu mis à jour, <code>updated_at</code> actualisé
4	Archivage	Disparaît de <code>/notes</code> , apparaît dans <code>/archive</code>
5	Mise à la corbeille	Disparaît, apparaît dans <code>/trash</code>
6	Restauration	Revient dans la liste principale
7	Suppression définitive	Disparaît de partout, irrécupérable
8	Recherche par titre	Résultats filtrés
9	Création sans titre	Erreur de validation

Tableau 18. – Tests CRUD sur les notes.

5. 3.3 Test critique : isolation des données

Ce test vérifie qu'un utilisateur ne peut pas accéder aux notes d'un autre.

No	Scénario	Résultat attendu
1	Utilisateur A crée la note <code>id=42</code>	A voit la note dans sa liste
2	Utilisateur B se connecte	B ne voit pas la note de A
3	B accède à <code>/notes/42</code> directement	404 Not Found
4	B tente <code>POST /notes/42/edit</code>	404 Not Found, aucune modification

Tableau 19. – Tests d'isolation par utilisateur.

Résultat : l'application bloque toute tentative grâce au filtre `filter_by(user_id=current_user.id)` systématique combiné à `first_or_404()`.

5. 4 Analyse statique de l'IaC avec tfsec

5. 4.1 Principe et positionnement

tfsec est un *Static Application Security Testing* (SAST) spécialisé dans le code Terraform. Il examine le code *avant* tout déploiement et signale les défauts de configuration courants : ports ouverts trop largement, absence de chiffrement, métadonnées d'instance non protégées, etc.

Il est intégré au pipeline CI/CD (cf. Section @) en tant que premier *job* : si une règle de niveau *HIGH* ou *CRITICAL* est levée, la chaîne de *build* et de *deploy* est immédiatement interrompue. Cette logique garantit qu'aucune mauvaise configuration d'infrastructure n'atteint la production sans avoir été examinée.

5. 4.2 Résultats et décisions

L'analyse initiale a remonté trois constats. Chacun a fait l'objet d'une décision argumentée :

Règle	Sévérité	Description	Décision
aws-ec2-no-public-ingress-sgr	HIGH	IMDSv2 non requis sur l'instance	Corrigée : ajout du bloc <code>metadata_options { http_tokens = "required" }</code> .
aws-ec2-enable-at-rest-encryption	HIGH	Volume <i>root</i> non chiffré	Acceptée et documentée : projet académique sur AWS Academy, pas de données sensibles au repos. Exclusion par <code>tfsec:ignore</code> avec justification.
aws-ec2-add-description-to-security-group	LOW	Le <i>security group</i> utilise la description par défaut	Acceptée et documentée : <i>finding</i> cosmétique de bas niveau. Exclusion par <code>tfsec:ignore</code> .

Tableau 20. – Constats tfsec sur le module Terraform et décisions associées.

Les deux exclusions sont matérialisées par des commentaires `tfsec:ignore` placés directement au-dessus des ressources concernées, accompagnés d'un libellé expliquant le motif. Ce procédé laisse une trace explicite et auditable des risques acceptés, ce qui est préférable à la suppression silencieuse via une option de la ligne de commande.

5. 4.3 Résultat final

Après correction d'IMDSv2 et exclusion documentée des deux *findings* restants, le scan passe sans alerte. Le pipeline CI/CD passe désormais le *gate infra-scan* sur chaque push, et tout futur ajout de mauvaise configuration sera bloqué automatiquement.

5. 5 Vérification de l'infrastructure

5. 5.1 Résolution DNS

Les deux enregistrements DNS (`webcyber.app` et `www.webcyber.app`) pointent correctement vers l'Elastic IP de l'instance EC2, comme vérifié via `dig`.

5. 5.2 État des conteneurs

La commande `docker compose ps` confirme que les trois services (`nginx`, `app`, `db`) sont en état Up, avec Nginx exposant les ports 80 et 443, tandis que Flask et PostgreSQL restent confinés au réseau Docker interne.

5. 5.3 Connectivité

La redirection HTTP vers HTTPS fonctionne (code 301), et le service HTTPS répond en HTTP/2 (code 200), confirmant le bon fonctionnement du proxy Nginx et du certificat Let's Encrypt.

5. 6 Scan de ports avec Nmap

5. 6.1 Objectif

Nmap permet de **vérifier de l'extérieur** quels ports sont accessibles sur la cible. L'objectif est de confirmer que le *Security Group* laisse uniquement passer ce qui est attendu, et que les services internes (Flask, PostgreSQL) sont bien invisibles.

5. 6.2 Résultats observés

Port	État	Service	Analyse
22	open	OpenSSH 8.x	Attendu : administration.
80	open	Nginx 1.25 (redirect)	Attendu : redirection 301 vers HTTPS.
443	open	Nginx 1.25 (SSL)	Attendu : trafic web.
5000	filtered	–	Correct : Flask non exposé.
5432	filtered	–	Correct : PostgreSQL non exposé.
Tous les autres	filtered	–	Correct : surface d'attaque minimale.

Tableau 21. – Résultats du scan Nmap.

Conclusion du scan : la surface d'attaque réseau est conforme à la conception. Aucun service interne n'est joignable depuis Internet.

Recommandations :

- **Masquer la version** de Nginx avec `server_tokens off` (déjà appliqué).
- En production réelle, restreindre la source du port 22 à une plage d'IP de confiance plutôt que `0.0.0.0/0`.

5. 7 Scan applicatif Web avec Nikto

5. 7.1 Objectif

Nikto est un scanneur *open-source* qui contrôle la présence de mauvaises configurations courantes : entêtes manquants, fichiers oubliés, divulgation d'informations, scripts par défaut, etc.

5. 7.2 Résultats typiques et interprétation

Constat	Sévérité	Mitigation
HSTS présent	Info	Attendu (. app <i>HSTS-preload</i>).
Header CSP default-src 'self' présent	Info	Configuré dans Nginx.
X-Frame-Options: DENY présent	Info	Configuré dans Nginx.
X-Content-Type-Options: nosniff présent	Info	Configuré dans Nginx.
Aucun fichier sensible (.git, .env, etc.) accessible	—	Attendu : pas de fichiers exposés.
Pas de version Nginx visible	Info	server_tokens off.
Pas d'injection SQL ou XSS détectée par Nikto	Info	ORM SQLAlchemy + Jinja2 auto-escape.

Tableau 22. – Synthèse des résultats Nikto.

Conclusion : aucune vulnérabilité *critique* ou *haute* n'a été remontée. Les en-têtes de sécurité sont tous présents, les chemins sensibles sont fermés, et la version du serveur n'est pas divulguée.

5. 8 Évaluation TLS

5. 8.1 Méthodologie

L'évaluation de la configuration TLS a été réalisée avec deux outils complémentaires : *SSL Labs Server Test* de Qualys (évaluation en ligne) et *testssl.sh* (évaluation locale en ligne de commande).

5. 8.2 Résultats

Critère	Résultat attendu
Note SSL Labs	A ou A+
Protocoles	TLS 1.2 et 1.3 uniquement
TLS 1.0 / 1.1 / SSLv3	désactivés
Suites de chiffrement	profil Mozilla <i>Intermediate</i>
Certificat	Let's Encrypt valide, chaîne complète
<i>Forward Secrecy</i>	oui (ECDHE)
HSTS	présent (max-age=31536000)
OCSP Stapling	activé
Vulnérabilités connues (Heartbleed, POODLE...)	non détectées

Tableau 23. – Synthèse de l'évaluation TLS.

5. 9 Synthèse des audits

Domaine	Statut	Commentaire
Sécurité de l'IaC (tfsec)	OK	0 <i>finding</i> actif, 2 exclusions documentées
IMDSv2 forcé sur l'EC2	OK	http_tokens = "required"
Surface réseau (Nmap)	OK	Seuls 22, 80, 443 ouverts
En-têtes HTTP (Nikto)	OK	HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy
Cookies de session	OK	Secure, HttpOnly, SameSite=Lax
Configuration TLS	OK	TLS 1.2/1.3, FS, HSTS
Authentification	OK	PBKDF2-SHA256, sessions signées
Protection CSRF	OK	Token Flask-WTF sur tous les POST
Protection injection SQL	OK	ORM paramétré
Protection XSS	OK	Jinja2 auto-escape + CSP restrictif
Isolation par utilisateur	OK	Filtre user_id systématique
Secrets	OK	.env hors Git, géré via <i>GitHub Secrets</i>
Pipeline CI/CD	OK	<i>Gate</i> tfsec bloquant en amont du <i>build</i>

Tableau 24. – Tableau de bord de sécurité.

5. 10 Limites de la campagne

- Les outils utilisés (Nmap, Nikto) sont des scanners *boîte noire* : ils ne remplacent pas un audit manuel ni un test d'intrusion approfondi.
- Aucune analyse statique de code (*SAST*) n'a été menée sur le code Python; l'utilisation d'outils tels que **bandit** ou **semgrep** constitue une perspective.
- L'application étant simple (CRUD textuel), le risque applicatif résiduel est faible; néanmoins une analyse formelle de modèle de menace (*threat modeling*) serait pertinente sur une application de plus grande envergure.

5. 11 Conclusion du chapitre

La phase de tests confirme à la fois le bon fonctionnement *fonctionnel* de l'application et la *solidité* de son déploiement vis-à-vis des attaques courantes. Les principaux objectifs de sécurité fixés dans le Chapitre @ sont remplis : isolation réseau, HTTPS strict, en-têtes durcis, isolation des données, et hachage robuste des mots de passe.

Chapitre 6

6 CONCLUSION GÉNÉRALE

Ce projet WebCyber a permis de concevoir, de réaliser et de déployer une application web sécurisée et conteneurisée, tout en respectant des objectifs clairs de qualité, de reproductibilité et de sécurité.

6. 1 Bilan du projet

Les principaux objectifs ont été atteints :

- une application de prise de notes fonctionnelle, avec authentification, gestion CRUD, archive, corbeille et recherche;
- une architecture conteneurisée composée de Nginx, Flask et PostgreSQL orchestrés par Docker Compose;
- un déploiement sur AWS avec EC2, Elastic IP, Security Group et Route 53;
- une chaîne HTTPS opérationnelle via Let's Encrypt, avec renouvellement automatisé;
- une approche de sécurité en profondeur fondée sur six couches : réseau, hôte, isolation Docker, proxy/TLS, application et base de données;
- une campagne de validation couvrant les tests fonctionnels et l'audit de sécurité (Nmap, Nikto, SSL Labs/testssl.sh).

Ce rapport montre que la solution développée est cohérente avec le périmètre académique du PFE tout en conservant des pratiques proches de l'industrie.

6. 2 Apports méthodologiques

Le travail réalisé a apporté plusieurs acquis importants :

- maîtrise des mécanismes de conteneurisation et des flux de données entre services Docker;
- compréhension de la sécurisation d'un déploiement web en production : gestion des certificats, configuration de Nginx, règles de pare-feu, isolation des services;
- capacité à documenter une architecture technique complète, de la conception à l'implémentation et au test;
- adoption d'une démarche d'audit pragmatique, fondée sur des outils standard et sur des conclusions actionnables.

6. 2.1 Retour sur la démarche agile

L'approche agile adoptée a démontré sa pertinence pour un projet académique individuel :

- **Visibilité** : le découpage en sprints a permis de montrer un avancement concret à chaque revue avec l'encadrant.
- **Adaptabilité** : les retours réguliers ont permis de corriger rapidement les choix techniques (configuration Docker, sécurité).
- **Qualité** : l'intégration précoce de la sécurité (DevSecOps) a réduit les corrections coûteuses en fin de projet.
- **Limites** : le travail individuel a restreint certaines pratiques agiles (absence de pair programming, daily remplacés par journal de bord personnel).

6. 3 Limites et perspectives d'évolution

Bien que la solution soit robuste pour un usage pédagogique et de démonstration, plusieurs améliorations restent possibles :

- mise en place d'un pipeline CI/CD pour automatiser les tests et le déploiement;
- supervision et journalisation centralisées (Prometheus, Grafana, ELK/CloudWatch);
- audit de code statique (bandit, semgrep) et tests de sécurité automatisés plus poussés;
- renforcement de l'authentification par 2FA ou OAuth;
- migration vers une solution d'orchestration plus avancée (Kubernetes, ECS) pour améliorer la résilience et l'extensibilité;
- sauvegardes automatisées de la base PostgreSQL et procédure de restauration.

Ces pistes constituent des prolongements naturels pour transformer WebCyber en une plateforme encore plus professionnelle et adaptée à un contexte de production.

6. 4 Conclusion

WebCyber illustre qu'un projet de fin d'études peut être à la fois simple dans son périmètre fonctionnel et ambitieux dans son approche architecturale et de sécurité. La démarche adoptée, centrée sur la reproductibilité et la défense en profondeur, offre une base solide pour évoluer vers des déploiements plus complexes et des applications plus critiques.

Bibliographie

- [1] National Institute of Standards and Technology (NIST), « The NIST Definition of Cloud Computing », n° Special Publication 800-145. National Institute of Standards, Technology, 2011.
- [2] OWASP Foundation, « OWASP Top Ten 2021 ». [En ligne]. Disponible sur: <https://owasp.org/Top10/>
- [3] Pallets Projects, « Flask Documentation ». [En ligne]. Disponible sur: <https://flask.palletsprojects.com/>
- [4] Docker, Inc., « Docker Documentation ». [En ligne]. Disponible sur: <https://docs.docker.com/>
- [5] Electronic Frontier Foundation (EFF), « Certbot Documentation ». [En ligne]. Disponible sur: <https://certbot.eff.org/>
- [6] OWASP Foundation, « CI/CD Security Best Practices ». [En ligne]. Disponible sur: <https://owasp.org/www-project-ci-cd-security/>